

NERV V2.1: The Self-Evolving Blockchain

Via Neural Weight Oscillation, Pure-Cryptographic Privacy, and Identity–Knowledge Separation

Version: 2.1

Date: December 15, 2026

Author: Rajeev Ragunathan

License: MIT / Apache 2.0

Abstract

NERV v2.1 is a layer-1 blockchain that simultaneously delivers privacy-by-default, horizontal scalability beyond one million transactions per second, post-quantum security from genesis, and *verifiable* per-block self-improvement — with no dependence on trusted hardware, no pre-trained AI models, and no unproven research assumptions.

NERV represents a ledger's state not as a Merkle tree but as a 512-byte neural state embedding: a 64-dimensional vector produced by a single-layer linear perceptron whose public weight matrix W compresses transaction activity into an exact, homomorphic fingerprint. Because the encoder is strictly linear, state updates are exact vector additions, provable in zero knowledge with roughly 50,000 circuit constraints — small enough to prove on a phone in under 50 milliseconds.

Three mechanisms distinguish v2.1 from all predecessor designs:

1. **Identity–Knowledge Separation.** The network's *identity* — the encoder W that defines state semantics — is frozen and versioned, changing only through slow governance epochs. The network's *knowledge* — a forecasting model F that predicts the next block's aggregate activity — is retrained every block by the Adam optimizer under Huber loss. A ledger's constitution must be static; its expectations may be alive. NERV separates the two, so learning can never corrupt the state it describes.
2. **Aggregate-Only Revelation.** No transaction's state-delta is ever individually decrypted by any node, ever. Deltas travel encrypted under a network-wide threshold key; block producers can only combine ciphertexts homomorphically,

and validators collectively reveal *one aggregate per block*. Public Pedersen commitments make that single revealed aggregate auditable by anyone — privacy without decryption oracles, using mathematics instead of hardware.

- 3. Forecast-Verified Self-Evolution. Every block, before the batch is decrypted, the network's forecaster commits (by hash) to a prediction of the aggregate. After decryption, the prediction is scored against reality. The committed residual — the network's public, non-repudiable *surprise* — drives both the weight update and an anomaly alarm. A promotion-gated market pays tokens only for challengers that beat the incumbent forecaster *out of sample*, on targets that were cryptographically sealed at commit time. The network's "getting smarter" is thus an auditable fact, not a slogan.

The result is a chain that is private by construction, quantum-resistant where it matters, fast by arithmetic, and honest about what it knows.

1. Introduction & Motivation

1.1 The Quadrilemma

The blockchain industry has learned to buy any two of the following four properties — speed, privacy, quantum security, adaptivity — at the price of the others:

Category	Examples	Throughput	Privacy	Quantum-safe	Adapts
High-throughput public chains	Solana, Sui, Aptos	100k–1M+	None	No	No

Privacy chains	Monero, Zcash	<100	Strong	No	No
Post-quantum initiatives	Penumbra-class designs	Varies	Medium	Yes	No
ZK rollups	zkEVM family	2k–10k	Partial	No	No

No production system offers all four. NERV exists to close that gap — and to do so without the two crutches that have historically made "intelligent blockchains" fail: trusted hardware enclaves, and neural networks so heavy that their proofs cost more than the transactions they describe.

1.2 What NERV Is, In Plain Language

At its core, NERV is a shielded payment ledger in the tradition of Zcash: users hold private notes (secret entries encoding ownership), spend them by publishing nullifiers (one-time tags that mark a note as spent without revealing which), and create new notes via commitments (hiding sealed boxes that only the recipient can open). Balances are never published; transactions prove, in zero knowledge, that value was conserved and double-spends are impossible.

On top of this standard, battle-tested foundation, NERV adds one unusual organ: a continuously maintained 512-byte summary of all economic activity, called the *state embedding*. It is produced by a linear map — a single-layer perceptron — and it exists for three reasons:

- Light-client efficiency: the entire state of a shard is fingerprinted by 512 bytes, so phones and hardware wallets sync almost instantly and verify inclusion forever.
- Homomorphic batching: because the map is linear, a transaction's effect on the summary is a small vector *delta*, and ten thousand transactions' deltas simply add up. The state update is one vector addition regardless of batch size — $O(1)$ in transaction count.
- A learning signal: the *sequence* of per-block aggregate deltas is a compact, public time series of the network's own economy — exactly the raw material a small forecasting model needs to adapt to shifting usage, load, and attack patterns in real time.

Everything else in this document is the machinery that makes those three properties safe: zero-knowledge circuits that prove deltas were computed correctly, a mixnet that hides who submitted what, threshold cryptography that ensures no one ever sees an individual delta, and a training loop that learns from every block without ever touching the ledger's semantics.

1.3 The First-Principles Shift: Why the Encoder Is Linear

Marrying machine learning to cryptography fails, structurally, at the point of non-linearity. Neural networks earn their power from non-linear activations (GELU, softmax), but zero-knowledge proof systems arithmetize computation over prime fields, where each non-linearity becomes lookup tables and thousands of constraints. A deep encoder forces millions of constraints per proof, GPU-scale provers, and — worst of all — only *approximate* additivity, which in a consensus system means validators disagree in the last decimal bits and the chain forks.

NERV's encoder is deliberately the smallest learner in existence: a single-layer perceptron,

$$f(x) = W \cdot x + b$$

with a public, frozen weight matrix $W \in \mathbb{R}^{64 \times N}$ ($N = 256$ input features) and bias handling described in §4. Linearity is not a compromise; it is the load-bearing decision, because a linear map is *exactly* homomorphic:

$$f(x + \Delta x) = (W \cdot x + b) + W \cdot \Delta x = f(x) + f(\Delta x)$$

Consequences:

- Exactness. State updates carry zero error. There are no error bounds to enforce, no drift to dispute, and no approximate arithmetic for consensus to disagree about. (Predecessor designs required error bounds of $\leq 10^{-9}$ – and discovered that one-part-in-a-billion agreement across thousands of independent validators is itself a consensus hazard.)
- Trivial proofs. Proving a delta was computed correctly is proving a dot product: multiplication and addition gates only. The circuit budget is $\sim 50,000$ constraints – under 50 ms of proving on a standard mobile CPU, ~ 400 -byte proofs.
- $O(1)$ state updates. Up to 10,000 transactions per block collapse into one aggregate delta, applied by 64 additions.
- No pre-training. The encoder does not need to *learn* to be homomorphic. Linearity gives it that property by definition, from the first block.

1.4 The Constitutional Principle: Identity Is Frozen, Knowledge Is Alive

The deepest lesson embedded in v2.1 – learned the hard way across two predecessor architectures (§1.5) – is this:

A blockchain's canonical functions must be fixed; only derived, advisory functions may learn.

If the state-defining parameters *learn*, two failure modes are possible, and both predecessor designs realized one each:

- The state can be defined as the model's *output*. Then updating the model rewrites the state's meaning, and every learned improvement is an act of institutional amnesia: old receipts no longer verify, and the supposedly irreversible ledger depends on the whims of training.
- The learning target can be defined as the model's *output*. Then the model is graded against its own reflection; the optimal solution is to stop learning; the training signal self-extinguishes; and "self-improvement" becomes a subsidy for rehearsing the identity function.

NERV v2.1 therefore splits what earlier designs fused, into two parameter sets with opposite temperaments:

	W — the Identity Layer	F — the Knowledge Layer
Role	Encodes transactions into state deltas; defines the embedding	Forecasts the next block's aggregate activity
Used by	Wallets, Halo2 circuits, state updates — <i>canonical</i>	Operations, sharding, anomaly flags, rewards — <i>advisory</i>
Update cadence	Frozen; governance epochs only; version-tagged	Every block, by Adam + Huber, deterministically
Failure blast radius	Catastrophic (state semantics corrupt)	Benign (a forecast is merely wrong, publicly)

Learns?

No — by design

Yes — this is the self-evolution

No consensus outcome, balance, proof, or receipt depends on F. The forecast is committed, scored, and either earns its keep or embarrasses itself — visibly, and without consequence to the ledger. This single separation is what makes per-block learning *safe enough to ship*.

1.5 Design Lineage: Three Attempts, One Principle

For reviewers and auditors, v2.1's shape is best understood as the third iteration of one idea.

Attempt 1 (v1.01): intelligence as canon. A 24-layer transformer compressed the ledger into a 512-dimensional embedding, attested inside hardware enclaves (TEEs). The curriculum was honest — trained on real ledger sequences — but two structural faults proved fatal. First, the *Epoch Paradox*: because the canonical state *was* the transformer's output, and the map was deliberately non-invertible for privacy, every model upgrade required re-encoding a state that could not be reconstructed. The ledger could remember or learn, never both. Second, TEE dependence: approximate homomorphism (error $\leq 10^{-9}$) required thousands of validators to run bit-identical binaries, which only attested hardware could guarantee — placing the privacy model's foundations in silicon vulnerable to side channels, not in mathematics.

Attempt 2 (v2.0): linearity as canon — but a mirror as teacher. Replacing the transformer with a linear perceptron eliminated the TEEs (exact homomorphism needs no attested execution), slashed the circuit from 7.9M to ~50K constraints, and removed pre-training entirely. But the per-block learning loop trained the encoder to predict a target that was itself computed from the encoder: the residual reduced to $(I - W) \cdot x$, whose optimum — W annealing toward the identity matrix — yields exactly zero loss, zero gradient, and zero further learning. The network's self-evolution extinguished itself by succeeding. Its reward pool paid for a convergence it could not help but complete.

Attempt 3 (v2.1, this document): both lessons enforced. Linearity retained (attempt 2's gift); the encoder frozen (attempt 1's disease excised); learning re-aimed at a target that is *cryptographically sealed at prediction time* (attempt 2's disease excised). The threshold-encrypted mempool, built for privacy, doubles as the exam proctor: the next

block's contents do not exist in plaintext anywhere on Earth at the moment the forecast must be committed. The learning signal is therefore exogenous, irreducible, and permanent.

1.6 Design Principles

P1 — Mathematics over hardware. No trusted execution environments anywhere in the protocol. Every privacy and correctness guarantee reduces to lattice assumptions, discrete logs (with stated, bounded roles), or symmetric crypto.

P2 — Identity frozen; knowledge live. Canonical functions (W, the note model, the circuit) are versioned and static between governance epochs. Learning is confined to derived, advisory models (F) whose failure is non-catastrophic.

P3 — Exogenous targets only. Nothing is ever trained on its own output. Every learning signal is revealed reality that was unknown (and cryptographically sealed) at prediction time.

P4 — Exactness in consensus. All consensus-relevant arithmetic is exact: linear algebra over 64-bit fixed point, with integer operations, specified rounding, and deterministic replay from genesis. No error bars, no tolerance windows, no votes on approximations.

P5 — Aggregate-only revelation. Individual plaintexts (deltas especially) are never decrypted by any single node. Threshold cryptography reveals sums; homomorphic commitments make the sums publicly auditable.

P6 — Out-of-sample rewards. Tokens are paid only for skill demonstrated on sealed targets after commitment — never for in-sample loss reduction, which is indistinguishable from curve-fitting.

P7 — Robust statistics throughout. Every learning surface uses Huber loss (bounded blame) and Adam (consistency-weighted, per-parameter steps) with gradient clipping: the network updates vigorously on ordinary events, cautiously on outliers, and never lets one loud event teach the lesson.

P8 — Honest scoping. The protocol claims adaptive *statistics*, not intelligence; anomaly flags are advisory; every non-post-quantum component is named, its role bounded to liveness or auditability, and a migration hook is specified.

2. Architecture Overview

2.1 The Three Layers

NERV is best understood as three stacked layers, each with its own primitives, guarantees, and failure semantics:

Layer 1 – Custody (the cryptographic ledger). Private notes, Pedersen commitments, nullifiers, and the nullifier/commitment trees. This layer owns *money*: who may spend what, and prevention of double-spends. It is deliberately conventional – the Zcash-proven UTXO model – because canonicity rewards fixed, boring, exact functions. Nothing in this layer ever learns anything.

Layer 2 – Identity (the state embedding). The public weight matrix W , the per-transaction delta computation $\delta_{tx} = W \cdot \Delta S + b_{tx}$, the 512-byte embedding e_t , and the Halo2 circuit that proves deltas correct without revealing them. This layer owns *compression and verifiability*: the whole economy of a shard, fingerprinted in 512 bytes, updatable in $O(1)$, provable on a phone. It is frozen (§4.4) so that its meaning never moves.

Layer 3 – Knowledge (the forecaster). The forecasting matrix F , the per-block commit–reveal–score–update loop, the challenger market, and the consumers of forecasts (shard load management, mempool ops, anomaly flags). This layer owns *expectations about the future*. It learns every block; it is wrong in public; nothing canonical depends on it.

Privacy and transport cut across all three: the PQ-Sphinx mixnet (network anonymity), the threshold-encrypted delta channel (aggregate-only revelation), and the signature/proof stack (post-quantum attestations).

2.2 The Life of a Transaction (Bird's-Eye View)

A complete walkthrough with business detail and formalism is Appendix A. In outline:

1. Construct – the wallet selects notes, builds outputs, computes the delta δ_{tx} under the current frozen W (with version tag), and proves correctness – plus non-zero-ness – in Halo2. It publishes a *public shell* (nullifiers, output commitments, Pedersen commitment to δ_{tx} , the proof) and seals the delta itself under the network's threshold key.

2. Route — the transaction traverses five staked mixnet relays wrapped in post-quantum onion encryption with timing jitter and cover traffic; no observer links sender to transaction.
3. Pool — the public shell enters the threshold mempool; every node can check the proof and nullifier freshness, but no node can read any individual delta.
4. Forecast & Seal — before decryption, every validator deterministically computes the forecaster's prediction of the aggregate and commits its hash into the block header. The exam is now locked.
5. Aggregate & Reveal — the producer *homomorphically combines* the encrypted deltas; a validator committee performs threshold decryption on the *aggregate only*; the single revealed sum is checked against the public commitments. No individual delta is ever exposed.
6. Update & Score — the embedding updates by vector addition; the committed forecast is scored against the revealed reality; the public residual — the network's surprise — is recorded; the forecaster's weights update via clipped Adam under Huber loss.
7. Finalize — validators vote on the executed state root (deterministic arithmetic plus proof verification), aggregating BLS signatures; finality in under 600 ms.
8. Split or Merge (rare) — a load-forecasting model may trigger deterministic shard bisection or, after sustained quiet, a merge.
9. Receipt — the user receives a ~3.5 KB Verifiable Delay Witness (VDW): a permanent, offline-verifiable proof of inclusion containing no transaction details.

2.3 Reader's Map: Vocabulary

Term	Meaning
------	---------

Private note	A secret record encoding ownership of an amount, spendable only by its holder
Commitment	A sealed, hiding "envelope" ($C = v \cdot G + r \cdot H$) that binds a value without revealing it
Nullifier	A one-time public tag whose revelation marks a specific note as spent, without identifying it
State embedding e_t	The 512-byte, 64-dimensional public fingerprint of a shard's cumulative activity
Delta δ_{tx}	The 64-dimensional vector a transaction adds to the embedding

W (identity layer) The public, frozen 64×256 matrix mapping transaction features to deltas

F (knowledge layer) The public, per-block-retrained forecaster of aggregate activity

DKG key A public key collectively generated by validators; no single party holds the secret

Threshold decryption Any t-of-n validator committee can jointly decrypt — never any single node

Aggregate-only revelation The protocol rule that only sums of deltas, never individual deltas, are ever decrypted

Sphinx mixnet Onion-routed relay network providing network-level anonymity

Halo2 proof	A succinct zero-knowledge proof (hundreds of bytes, milliseconds to verify)
VDW	Verifiable Delay Witness: the user's permanent inclusion receipt
Adam / Huber	The optimizer (consistency-weighted steps) and loss (bounded blame) that train F

2.4 Threat Model and Guarantees

Adversary	Sees	Guarantee NERV provides
Global passive network observer	Packet timings, sizes, IPs	Sphinx onion routing, exponential jitter ($\mu = 100$ ms, $\sigma = 200$ ms), and learned cover traffic prevent sender–transaction linkage (k-anonymity target $>10^6$)

Mempool observer	Public shells (nullifiers, commitments, proofs) – but no amounts, no parties, no individual deltas	Shielded-pool cryptography (Zcash-class) plus sealed delta channel; nothing in the mempool links users, amounts, or counterparties
Block producer	Shells, batch composition	Cannot read any individual delta (aggregate-only revelation); cannot reorder for profit (sums are order-invariant); inclusion-steering is bounded by aggregate scale and statistically visible in committed residuals
<1/3 Byzantine validators	Their own views	BFT safety on the executed root; rogue weight updates impossible (F is deterministic replay, not voted); invalid partial decryptions carry pairing-based proofs and are slashed

Quantum adversary (future) Everything recorded

Fund security (signatures, KEM) is
lattice-based from genesis; BLS and
Pedersen roles are explicitly
bounded to liveness/auditability
with migration hooks (§11)

Adversarial data (MEV
spikes, spam cascades)

The aggregate stream

Huber loss caps per-dimension
blame; gradient clipping caps
update norm; anomaly flags fire on
tail residuals; the ledger itself never
moves — only expectations do

What NERV does not claim. The forecaster is adaptive statistics, not intelligence. Anomaly flags are advisory inputs to operations and review, not automatic slashing triggers. Delta-level privacy is computational (a rank-64 projection) rather than information-theoretic, and is parameterized by W 's conditioning — reviewed at every governance epoch. These limits are stated here because a protocol that overclaims is a protocol that will be trusted once.

3. The Accounting Model: Private Notes, Commitments, Nullifiers

3.1 Design Intent: Boring Where the Money Lives

NERV's custody layer is conventional on purpose. Before any neural machinery appears, the question "who owns what, and how do we prevent double-spends?" must be answered by the most battle-tested construction in shielded finance: the note/nullifier model pioneered by Zcash and audited by a decade of adversarial production use.

Principle P2 (identity frozen) applies most strictly here: nothing in the custody layer ever learns, adapts, or evolves. Its functions are fixed at genesis.

The reader should hold one framing throughout this document: NERV's ledger is *two parallel structures*. The first — this section — is the authoritative ledger: it tracks ownership exactly, prevents double-spends, and enforces conservation of value. The second — Section 4 — is a derived public index: a 512-byte lossy fingerprint of economic activity. The authoritative ledger never consults the index. Money lives in cryptography; only summaries live in linear algebra.

3.2 Private Notes

A private note is the atomic unit of ownership: a secret record that binds an amount to a spending authority. The note plaintext is never published. Its fields:

Field	Description
v	Value in nano-NERV (integer, $0 < v \leq 2^{60}$; the cap reserves guard bits for arithmetic)
p	Unique note nonce (256-bit random, chosen by the sender at creation)

d Recipient's diversified delivery public key (one of many one-time addresses per wallet)

memo Optional encrypted memo (up to 80 bytes)

Address and key hierarchy. A wallet derives everything from a 256-bit master seed (BIP-39 compatible, PBKDF2-HMAC-SHA512): a spending key `sk_spend`, a note-derivation key `nk`, and detection keys, via domain-separated BLAKE3-KDF expansion. Recipients publish diversified addresses — cheap, unlimited, unlinkable delivery points — so each note's delivery key `d` can be unique. A viewing/detection key can be delegated for regulated wallets without ever exposing spending capability: detection keys reveal *which* notes are yours; only `sk_spend` can move them.

Note encryption. The note plaintext is sealed to the recipient using hybrid PQ encryption:

- `(ss, ct) = ML-KEM-768.Encapsulate(d)` — a lattice key encapsulation to the delivery key
- `k = BLAKE3-KDF(ss, "nerv.note")`
- `enc_note = ChaCha20-Poly1305.Encrypt(k, note_plaintext)`

The encrypted note is published on-chain as an opaque blob. Recipients discover incoming notes by trial-decrypting new blobs with their detection keys — standard, private, and requiring no interaction with the sender or any server.

3.3 Note Commitments and the Commitment Tree

To spend a note later, the spender must prove it exists — without revealing which note. Each note is therefore published only as a commitment:

`cm = BLAKE3("nerv.cm" || v || p || d || r)`

where r is a fresh 256-bit blinding factor chosen by the sender. This construction is *hiding* (r makes cm computationally indistinguishable from random) and *binding* (finding two distinct note contents with the same cm is a BLAKE3 collision — infeasible for classical and quantum adversaries alike; hash commitments are deliberately preferred here over elliptic-curve commitments, whose binding would fall to Shor's algorithm).

All commitments live in an append-only note commitment tree (NCT): a BLAKE3 Merkle tree of depth 32 (capacity ≈ 4.3 billion notes per shard), with its root committed in every block. Membership in the tree is what makes a note spendable; the membership *proof* is revealed only inside a zero-knowledge circuit (Section 5), never in public.

3.4 Nullifiers and the Spent Set

Spending must be public *in effect* (a note can never be spent twice) yet private *in identity* (no one learns which note was spent). The instrument is the nullifier:

```
nf = BLAKE3-ToField("nerv.nf" || nk || p)
```

Derived from the spender's secret note key and the note's unique nonce, the nullifier is a one-time pseudorandom tag. It cannot be linked back to cm (both p and nk are hidden — one by the commitment's blinding, the other by secrecy). The protocol maintains a public, append-only nullifier set (a Merkle-accumulated structure whose root is committed in every block). The rule is absolute and enforced by plain public check:

A transaction is invalid if any of its nullifiers already appears in the nullifier set; on inclusion, they are added.

Double-spend prevention is thereby reduced to a set-membership lookup — exact, deterministic, and independent of every other component in the system.

3.5 The Transaction: Public Shell and Sealed Delta

A NERV transaction consists of a public shell (visible to all, gossipable, checkable by any node) and a sealed delta (encrypted under a network collective key; never individually decrypted by anyone — Section 6).

Public shell (cleartext)

Sealed (threshold-encrypted)

Nullifiers of all input notes

The 64-dimensional delta δ_{tx}

Output note commitments $cm_1 \dots cm_k$

Encrypted output notes enc_note (opaque blobs)

Declared fee (integer nano-NEER)

ZK proof π (Section 5)

Weight-version tag (which W was used)

Expiry height

The fee is public by design: validators must price inclusion and anti-spam without reading contents. Everything else about amounts, counterparties, and notes is private or sealed.

Spend and output construction. The wallet selects input notes covering value + fee, creates output notes (recipient's and change), computes the scalar conservation identity (inputs = outputs + fee), derives nullifiers, computes δ_{tx} under the current frozen encoder W (Section 4), proves everything in zero knowledge, seals δ_{tx} , and routes the shell through the mixnet (Section 6). All of this is client-side; the network sees only the shell.

3.6 Value Conservation and Range Proofs

The zero-knowledge proof enforces the ledger's physics:

1. Conservation: $\sum \text{input values} = \sum \text{output values} + \text{fee}$, over all notes touched.
2. Ownership: For each input note, the spender knows sk_{spend} for a commitment cm found in the NCT (membership via Merkle path in-circuit), and the published nullifier is correctly derived from that same note (binding the spend authorization to the nullifier).
3. Freshness: Nullifiers are correctly formed from the note's nonce p .
4. Range: Every value lies in $[0, 2^{60}]$ — proven in-circuit with Pedersen-gadget range proofs (the one place Pedersen algebra appears in NERV: purely as an internal circuit gadget, inside the SNARK, where its classical assumptions are irrelevant because the outer proof is what the world verifies).
5. Delta correctness: $\delta_{tx} = W \cdot \Delta S$ was computed with the tagged version of W over a well-formed feature vector (Section 4.2), and the sealed ciphertext encrypts exactly that δ_{tx} (Section 6.4).

If any check fails, the proof does not verify, and no honest node will include the transaction. Conservation is thus enforced *mathematically per transaction* — not statistically, not by aggregate audit, not by machine learning.

3.7 Genesis and the Shielded Supply

The entire supply (10,000,000,000 NERV; Section 13) is created at genesis as an initial distribution of private notes committed into the NCT at height 0, per the published allocation. No inflation follows: the fee market and the Useful-Work pool (paid from genesis reserves, not emission) are the only validator compensation. Optional transparent→shielded minting windows may be enabled by governance; shielded→transparent burns are always available (a burn is simply a note spent with no output note and a public burn commitment, verifiable by anyone).

3.8 What This Layer Deliberately Does Not Do

- It does not learn, adapt, or reference any model. A total corruption of Section 4's machinery cannot mint, steal, unlock, or reorder a single note.
- It hosts no general smart contracts at genesis. NERV v2.1 is a payments network; the note model's extension points (shielded pool contracts) are a governance-scoped roadmap item, not a launch dependency.
- It decrypts nothing. Every plaintext that must ever exist for a user is produced by that user's wallet; every value the *network* needs is either public in the shell or revealed only as an aggregate (Section 6).

4. The Identity Layer: The Neural State Embedding

4.1 The Embedding and the Composite State Root

Each shard maintains a state embedding:

$e_t \in (\mathbb{Z}/2^{64})^{64}$ — a 64-dimensional vector of 64-bit words; exactly 512 bytes.

The embedding is an *accumulator*: it is updated per block by vector addition modulo 2^{64} (wraparound arithmetic — the accumulator is a closed group, so there is no overflow, no

saturation, and no drift; there is only exact, replayable addition). It is public, and its entire history is recomputable by anyone from the chain of revealed aggregates.

Crucially, the embedding is *not* the whole state, and its hash is *not* alone the state root. The canonical state of a shard is the tuple:

$S_t = (e_t, nct_root_t, nullifier_root_t, params_root_t, model_root_t, validator_root_t)$

and the block commits the composite root:

$h_t = \text{BLAKE3}(e_t \parallel nct_root_t \parallel nullifier_root_t \parallel params_root_t \parallel model_root_t \parallel validator_root_t)$

where $params_root$ commits the active weight version and governance parameters, and $model_root$ commits the forecaster state (Section 7 — which, being a deterministic replay of public data, is itself consensus state). The composite root eliminates an entire class of ambiguity: validators, light clients, and receipts all reconcile against the *same* single hash of *everything* canonical.

4.2 The Encoder W and the Feature Vector

The encoder is a single-layer perceptron without non-linearities — a public matrix:

$W \in (\mathbb{Z}/2^{16})^{64 \times 256}$ — 64 output dimensions $\times$ 256 input features, 16-bit signed fixed-point weights (1.15 format), quantized and hash-committed at each governance epoch.

A transaction's feature vector $\Delta S \in \mathbb{Z}^{256}$ is constructed client-side from the transaction itself:

Features	Indices	Content
Account slots	0–223 (224 slots)	For each input: $-v$ at slot(own spending address). For each output: $+v$ at slot(recipient's delivery address). Slots are public hashes: $\text{slot}(\text{addr}) =$

BLAKE3("nerv.slot" || addr) mod 224 — deterministic,
client-computable, provable in-circuit via lookup

Volume rail	224	Total output value (nano-NERV)
Fee rail	225	Declared fee
Count rail	226	1 (marks a transaction event)
Log-magnitude rail	227	$\lfloor \log_2(1 + v) \rfloor$ via circuit lookup (features may be non-linear transforms of transaction data; only the <i>map</i> W must be linear — this gives downstream models a scale-stationary signal)

Type/timestamp rails 228–255

Transaction-type flags, coarse time-of-epoch buckets
(public, not private timing)

Two properties of this layout deserve emphasis. First, sparsity: an ordinary transfer activates at most 6 account slots plus a handful of rails, so the dot product $W \cdot \Delta S$ reduces to selecting and scaling at most a dozen of W 's 256 columns — the circuit does this with lookup-gated column selection (Section 5), roughly 1,500 multiplication constraints, versus 16,384 for the dense worst case. Second, collisions are harmless: two different addresses may hash to the same slot. The feature vector feeds a lossy public index, not the ledger; ownership and solvency live entirely in Section 3. A slot collision slightly blurs the index and nothing else.

The transaction's delta is then:

$$\delta_{tx} = W \cdot \Delta S \in (\mathbb{Z}/2^{64})^{64}$$

computed by the wallet, proven correct in zero knowledge, and sealed for aggregate-only revelation.

4.3 The Exact Transfer Homomorphism

Linearity is the load-bearing wall. The encoder has no bias term and no non-linearity, so for any two feature vectors:

$$W \cdot (x + y) = W \cdot x + W \cdot y \text{ — exactly, by algebra, with error identically zero.}$$

There is nothing to train, nothing to approximate, and no error bound to dispute. The block-level consequence:

$$\Delta_B = \sum_{tx \in B} \delta_{tx} \text{ and } e_{t+1} = e_t + \Delta_B \pmod{2^{64}}$$

A batch of up to 10,000 transactions updates the state in exactly 64 additions — $O(1)$ in transaction count — and because vector addition is order-invariant, no participant can profit by reordering the batch. This is the property that funds the entire architecture: it is why the circuit is $\sim 50,000$ constraints instead of millions, why no hardware attestation is needed to keep validators in agreement, and why a phone can prove and verify state transitions in milliseconds.

4.4 The Bias Question

Predecessor designs included a global bias b and a per-transaction bias b_{tx} derived from nullifiers. NERV v2.1 retires both from the canonical state, for two reasons:

1. Accumulated bias is an unbounded random walk. A per-transaction pseudorandom 64-dimensional bias, summed over billions of transactions, injects hash noise into the embedding whose variance grows without bound — and modulo wraparound eventually scrambles the signal it was meant to protect. A state accumulator should accumulate *economy*, not entropy.
2. Noise poisons the learning signal. The knowledge layer (Section 7) is trained on revealed aggregates. A hash-noise component in every aggregate is, by construction, unlearnable; training against it wastes capacity and degrades anomaly detection.

The per-transaction nonce survives where it belongs — *inside the proof*: the nullifier-derived value serves as an in-circuit binding challenge linking the proof to its specific nullifiers (replay protection at the proof level). The forecasting layer has its own bias parameter c , living entirely in the knowledge layer where it can be retrained freely. Nothing in the canonical map needs a bias; a linear encoder of *increments* is homogeneous by design.

4.5 The Null-Space Defense: Injectivity Instead of Noise

A linear map has a null space: vectors $\Delta S \neq 0$ with $W \cdot \Delta S = 0$. In NERV v2.1 the note layer owns solvency (P8: honest scoping), so a null-space transaction cannot *steal* — conservation and ownership are enforced independently of the embedding. What injectivity protects is the embedding's role as a complete public index: the guarantee that every economically meaningful event — every genuine transfer — provably moves the fingerprint, and therefore that light clients and the forecaster never miss real activity.

Rather than defending with noise (v2.0's approach, which polluted the state; §4.4), v2.1 defends with structure:

- The circuit enforces that every ΔS lies in the *admissible support*: at most 6 active account slots with nonzero amounts, plus the rails block. Arbitrary adversarial feature vectors are not provable.
- The account-slot block of W is required to be injective on the admissible support: no nonzero admissible ΔS maps to zero. A sufficient condition is that any 13 of

the 224 slot columns are linearly independent (a *spark* condition; $k_{\text{max}} = 6$ active slots $\Rightarrow$ differences involve at most 12 columns).

- Achieving and certifying it: W is generated deterministically by verifiable expansion from a governance randomness beacon (BLAKE3-XOF of a public, multi-party beacon transcript). A generic random 64×224 matrix has spark 65 — comfortably above 13 — with overwhelming probability; each governance epoch re-verifies the bound by randomized rank checks over the generation transcript before adoption. W is never hand-tuned, so its injectivity is a mathematical property of public randomness, not a trusted party's craftsmanship.
- As defense in depth, the circuit additionally asserts $\delta_{\text{tx}} \neq 0$ as a vector (~ 64 cheap constraints) — a belt over the structural suspenders.

The result: *every provable transaction moves the public state*, with zero noise added and zero per-transaction cost beyond a handful of gates.

4.6 Batch Minimums and Aggregate Privacy

The embedding is public, and per-block revealed aggregates are sums over batches. A batch of one would reveal a single transaction's delta direction — a 64-dimensional projection of its feature vector. NERV therefore enforces a minimum reveal batch:

- Sealed deltas are aggregated and revealed only in batches of at least $B_{\text{min}} = 64$ transactions (governance-tunable); pending sealed deltas roll forward to the next block otherwise.
- Batches are capped at 10,000 for decryption-ceremony latency.

At $B_{\text{min}} = 64$, a revealed aggregate mixes 64 random slot-difference directions scaled by amounts; the marginal information about any single transaction is bounded and further obscured by the lossy 64-dimensional projection itself. A formal leakage analysis of (W, B_{min}) is a standing governance review item (Section 11), and B_{min} is the knob that trades reveal latency against aggregate anonymity. NERV states this honestly: the embedding's privacy is *computational and aggregate*, one tier below the note layer's — which is precisely why the note layer, not the embedding, is authoritative.

4.7 Weight Versioning and Governance Epochs

The encoder W is frozen between epochs (P2). It changes only through a slow, public governance ceremony:

Epoch checklist. (1) A fresh randomness beacon produces candidate W' . (2) The epoch certificate verifies: quantization format; weight norms; the spark/injectivity bound (§4.5); column–rail independence. (3) Governance (stake-weighted vote of note-holders and validators, Section 13) adopts W' with an activation height. (4) `params_root` updates to commit $\text{Hash}(W' \parallel \text{version})$.

The Amnesia Rule — the state is never re-encoded. At the epoch boundary, e_t carries forward *unchanged*. New deltas thereafter are computed under W' and simply added on. The embedding becomes a layered record — each epoch's increments written in that epoch's coordinates — which is correct because the accumulator only ever needs addition to be exact, and each delta proof pins its own W -version. This one rule is the difference between NERV v2.1 and the first-generation architecture it descends from: the chain upgrades its encoder without an act of amnesia. Old receipts remain valid forever (they verify against the W -version committed inside them); light clients need W' only for future deltas.

Downstream consequences, specified once here and honored later: the forecaster is re-initialized at every epoch boundary (its optimizer state resets; it may warm-start only on post-boundary aggregates — Section 7), because a change of W changes the coordinate system of the learning target.

4.8 What the Embedding Is — and Is Not

Is: a 512-byte public accumulator; a lossy, linear, injective-on-admissible-support index of economic activity; $O(1)$ -updatable; exactly provable in zero knowledge over $\sim 50,000$ constraints; the substrate on which the knowledge layer learns (Section 7).

Is not: the ledger of record (notes and nullifiers are); a source of authority for any balance; a privacy mechanism of the same tier as the note layer; a learned artifact (it is the *frozen* half of a deliberately split brain).

This is the crown safety property of the identity layer, and it closes the section:

Corrupting W — even totally, even catastrophically — can degrade the quality of NERV's public index. It cannot mint a note, spend a note, double-spend a note, or unbalance the conservation equation. Money lives in cryptography; the embedding lives in linear algebra; and only the second of those is ever allowed to have an opinion.

5. LatentLedger Lite: The Zero-Knowledge Circuit

5.1 What a Transaction Must Prove

Every NERV transaction arrives with a zero-knowledge proof bundle π that convinces any verifier — a relay, a validator, a phone — of the following statements, while revealing nothing beyond the public shell:

#	Statement proven	Why it matters
1	Each input note commitment exists in the note commitment tree (Merkle membership at the referenced root)	The spent value exists
2	The spender knows the spending authority for those notes	Only owners can spend
3	Each published nullifier is correctly derived from the spent note's secret key and nonce	Double-spend tags are authentic

4	$\Sigma \text{ inputs} = \Sigma \text{ outputs} + \text{fee}$	Value is conserved (Section 3.6)
5	Every value lies in $[0, 2^{60}]$	No overflow, no negative money
6	The feature vector ΔS is well-formed and on the admissible support (≤ 6 active account slots; correct volume/fee/count/log rails)	The encoder is fed honest inputs (§4.5)
7	$\delta = W_{\tau} \cdot \Delta S$ under the tagged, frozen encoder version	The delta is correct
8	$\delta \neq 0$	Every provable transaction moves the public index

9	$C = \text{PedVec}(\delta; r)$ — the public Pedersen vector commitment binds the sealed delta	Enables public aggregate audit (§6.3)
---	--	---------------------------------------

10	The sealed ciphertext ct encrypts exactly the committed pair $(\delta; r)$	No grieving via malformed encryptions
----	---	---------------------------------------

The wallet additionally includes, in the witness, a nullifier-derived binding challenge — the hash of the transaction's own nullifiers — woven into the proof's challenge computation. This links the proof inseparably to its specific nullifiers: replaying a proof against different nullifiers, or transplanting it to another transaction, breaks verification (the replay defense inherited from earlier designs, relocated from the state — where it accumulated as noise, §4.4 — to the proof, where it costs nothing).

5.2 One Bundle, Two Sub-Proofs

The bundle $\pi = (\pi_{\text{spend}}, \pi_{\delta})$ splits along a natural seam, connected by a shared public feature digest (a BLAKE3 hash of ΔS committed inside both proofs):

- π_{spend} — the shielded-payment proof: statements 1–5 plus the feature digest. This is the Zcash-class machinery: note membership, ownership, nullifier derivation, conservation, ranges.
- π_{δ} — the LatentLedger Lite core: statements 6–10 against the committed digest. This is NERV's own contribution, and it is deliberately tiny: a sparse dot product, a non-zero check, a commitment, and a sealing-consistency check.

The split earns its keep twice. It isolates the protocol-specific delta logic (which governance epochs may touch) from the custody logic (which must never change), and it lets the delta core — tens of milliseconds of proving on a modern phone — be audited, benchmarked, and upgraded independently of the heavier spend machinery. The full bundle proves client-side in a few hundred milliseconds, and wallets begin proving the

moment the user signs — while the packet is still traversing the mixnet (median transit ≈ 1.3 s, §6.2). User-perceived proving latency is therefore approximately zero.

5.3 Circuit Architecture

Both sub-proofs are PLONKish circuits (the spreadsheet model: fixed columns carry the rules, advice columns carry the wallet's secrets, instance columns carry the public shell), composed from six reusable chips:

Chip	Job	Dominant technique
Merkle chip	Verify 32-deep BLAKE3 membership paths	BLAKE3 compression via lookup tables (16-bit word lookups keep each level ~ 450 constraints)
Nullifier chip	Derive nf from witness keys/nonces	BLAKE3-ToField with shared lookup tables
Sparse encoder chip	Compute $\delta = W \cdot \Delta S$	Lookup-gated column selection: only ≤ 6 slot columns + 5 rails are selected from the fixed

public matrix, scaled by amounts — ~2,000
constraints instead of a dense 16,384

Range chip	60-bit value ranges	Lookup-based windowed range proofs
Pedersen chip	Verify $C = \text{PedVec}(\delta; r)$	Windowed multi-scalar multiplication of 65 generators (64 delta dimensions + blinding), ~11,000 constraints
Sealing chip	Verify ct is a well-formed encryption of $(\delta; r)$	The lattice ciphertext's linear structure (§6.3) makes this a system of modular linear equations — checked, not solved

The spark/injectivity property of W (§4.5) is deliberately not proven in-circuit: it is a property of the matrix itself, certified once per governance epoch. The circuit needs only the admissibility check (6) and the non-zero check (8) — the belt over structural suspenders.

5.4 Constraint Budget

For the typical transfer (2 inputs, 2 outputs):

Component

Constraints

Merkle membership (2 paths × 32 levels)

~29,000

Nullifier derivation (2)

~1,400

Conservation + ranges (4 notes)

~800

Feature construction + digest

~500

Support admissibility + log rail

~1,000

Sparse encoder $\delta = W \cdot \Delta S$	~2,000
--	--------

$\delta \neq 0$	64
-----------------	----

Pedersen vector commitment	~11,000
----------------------------	---------

Sealing consistency	~5,000
---------------------	--------

Wiring, challenges, blinding rows	~1,200
-----------------------------------	--------

Total bundle	~52,000
--------------	---------

The delta core π_δ alone is ~20,000 constraints — tens of milliseconds to prove on a modern phone. The full bundle is ~52,000 — a few hundred milliseconds on the same phone, comfortably hidden inside mixnet transit. On the constraint-per-dollar ledger this

is where the linear encoder pays its rent: the equivalent statement for a deep encoder would exceed several million constraints.

5.5 Recursion and Batch Verification

Per-block, the protocol does not verify 10,000 proofs. Proofs are Nova-folded: each transaction's bundle folds into a running accumulator, and the block carries a single recursive proof (a few hundred bytes to ~1 KB, constant in batch size) attesting that every bundle in the batch verified. Validators and light clients therefore perform one verification per block (< 10 ms), and the chain's verification load is $O(1)$ in throughput — the property that keeps light-client sync under 100 KB of headers, forever.

Setup posture: the proving system uses Halo2 with a universal, updatable structured reference string (one shared ceremony, no per-circuit toxic waste), with the transparent IPA configuration available as an alternative where ceremony-free deployment is preferred.

5.6 Honest Scoping

The zero-knowledge proof system is not post-quantum: Halo2's security rests on elliptic-curve discrete logarithms. NERV's posture is deliberate and bounded (P8): fund *authorization* rests on Dilithium-3 and value *custody* on hash commitments — both quantum-resistant — while the SNARK's roles are succinct verification, privacy, and auditability. A future quantum adversary who breaks the proof system degrades privacy and cheap verification; they cannot forge spending authorization or rewrite the nullifier set. A migration hook to hash-based proof systems (STARK family) is specified in Section 11.

6. The Privacy Stack: Mixnet Anonymity and Aggregate-Only Revelation

6.1 Four Walls

NERV's privacy is not one mechanism but four independent walls, each defending a different secret:

Wall	Protects	Mechanism	Section
Custody	Amounts, parties, note contents	Note encryption, commitments, nullifiers, ZK proofs	3, 5
Network	The link between a sender's network identity and their transaction	PQ-Sphinx mixnet	6.2
Delta channel	Individual transaction deltas — forever	Sealed, linearly homomorphic threshold encryption	6.3
Revelation	Per-block aggregates from single-transaction leakage	Batch minimums + lossy linear projection	6.5, 4.6

The design rule across all four (P5): plaintexts that matter are produced by wallets, never by the network. The network's only plaintext outputs are per-block aggregates.

6.2 The PQ-Sphinx Mixnet

What it protects — stated precisely. A transaction's public shell is designed to be publishable: it contains no addresses, amounts, or linkable metadata. What the mixnet protects is the *association* between the shell and the IP address that submitted it. Without this wall, a global observer would correlate timing and origin across every submission and de-anonymize users statistically, regardless of the cryptography underneath.

Operation. Wallets select 5 relays from the on-chain relay registry — staked, slashable operators whose Dilithium-attested ML-KEM-768 public keys are committed on-chain. Selection is VRF-seeded per transaction (deterministic for the wallet, unpredictable to observers), diversified by geography and operator. For each hop i , the wallet encapsulates a fresh shared secret $ss_i = \text{ML-KEM-768.Encapsulate}(pk_i)$, derives a domain-separated key $k_i = \text{BLAKE3-KDF}(ss_i, \text{"sphinx"} \parallel i)$, and wraps the next layer in ChaCha20-Poly1305. Each relay decapsulates, peels exactly one layer, learns only the next hop, and forwards. The final relay submits the shell to the threshold mempool; no relay ever sees plaintext — there is none to see, only layers.

Packet discipline. All packets are uniform 8 KB Sphinx-format messages (fixed size defeats volume analysis; the header is indistinguishable from random). Replay is defeated by per-hop MAC verification with monotonically tagged sequence numbers inside the Sphinx header.

Timing defense. Each relay holds every packet for a delay drawn from an exponential distribution (mean 100 ms, within the deployment's jitter envelope of $\mu = 100 \text{ ms} / \sigma = 200 \text{ ms}$, hard-capped at 500 ms). End-to-end ingress latency: median $\approx 1.3 \text{ s}$. This is the price of network anonymity, paid once, at submission — it does not affect block time, which is governed by the mempool pipeline, not the mixnet.

Cover traffic. Relays emit dummy packets — valid Sphinx messages addressed to other relays and discarded at their destination layer — shaped by a local learned traffic model running per-relay: the same linear-learner pattern as the network forecaster (Section 7), trained on the relay's own observed traffic to mimic its size and timing distribution. These models are knowledge-layer class: local, non-canonical, never consensus inputs. Honest limitation (P8): the network can slash provable misbehavior (dropping, malformed forwarding) but cannot slash *poor modeling*; cover-traffic quality depends on

relay competence, which stake and reputation markets price but do not guarantee. Relay software and dummy-generation specifications are open-source and audited for this reason.

Stated targets. k -anonymity $> 10^6$ against a passive global adversary; $> 10^5$ against an active adversary controlling $< \frac{1}{3}$ of relays. These are design targets under sustained operation with a healthy registry, not unconditional guarantees.

6.3 The Sealed Delta Channel: Aggregate-Only Revelation

This is the mechanism that replaces every predecessor's decryption oracle — trusted hardware in the first generation, a producer-decrypts-the-batch ceremony in the second — with arithmetic.

Sealing. Alongside the public shell, each wallet publishes a sealed ciphertext ct and a public Pedersen vector commitment $C = \text{PedVec}(\delta; r) = \sum \delta_i \cdot G_i + r \cdot H$ — one 48-byte point binding the entire 64-dimensional delta with fresh blinding r (perfectly hiding, computationally binding; its role is auditability, bounded per Section 11). The ciphertext encrypts the *pair* $(\delta; r)$ under the network's collective public key using a lattice-based linearly homomorphic threshold cryptosystem (module-LWE family): ciphertexts are additively combinable, and no single node can decrypt anything.

Why the ciphertext itself must be homomorphic — the key invariant. When the block is assembled, the producer (or any node — the operation is public arithmetic) computes the aggregate ciphertext:

$$ct_B = \sum_{tx \in B} ct_{tx}$$

Because the cryptosystem is linearly homomorphic, ct_B is a valid encryption of the *sums only*: $(\Delta_B; R) = (\sum \delta_{tx}; \sum r_{tx})$. Nothing else is recoverable from it — not by the producer, not by any committee member, not by their collusion short of the full threshold.

Revelation. A committee of $t = \lceil 2n/3 \rceil$ validators, holding DKG-shared decryption shares, computes partial decryptions of ct_B — each accompanied by a non-interactive zero-knowledge proof of correct partial decryption (invalid partials are rejected before combination and the sender slashed). Lagrange combination of t valid partials yields exactly one thing: the aggregate pair (Δ_B, R) . The ceremony is re-keyed every governance epoch, and key shares are proactively erased afterward — a future

compromise of the committee cannot retroactively decrypt historical individual ciphertexts, because the keys no longer exist.

Public audit. The revealed pair is checked by anyone against the shell commitments:

$$\Pi_{tx} \in B \quad C_{tx} = \text{PedVec}(\Delta_B; R)$$

If the equation holds, the revealed aggregate is exactly the sum of the values each wallet committed — the reveal is complete (nothing withheld) and honest (nothing fabricated). If it fails, the batch is quarantined, partial decryptions are individually audited via their proofs, and the faulty party is identified and slashed. This single equation is what makes "trust the committee" unnecessary: the committee's output is self-auditing public arithmetic.

What this achieves, in one sentence: the producer orders ciphertexts; the committee opens one sum; the world checks one equation; and no individual delta is ever plaintext to anyone, at any time, in any component's memory — the guarantee that earlier architectures attempted to obtain from hardware, obtained here from linearity.

6.4 The Threshold Mempool

Shells (with sealed deltas and proofs) gossip over GossipSub among validators. Admission checks are all public or zero-knowledge: format, fee sufficiency (fees are public by design), proof verification, nullifier freshness against the public nullifier set (double-spend detection *before* inclusion — nullifiers being public pseudorandom tags makes this a plain set lookup), and anti-spam rate limiting (with an optional proof-of-work escalation under attack). Batching follows three rules: batches contain between $B_{\min} = 64$ and 10,000 sealed deltas (§4.6); expired transactions are dropped and their nullifiers never posted; and because state updates are sums — order-invariant — the producer's ordering power carries no extractable value. Inclusion priority is by fee and age, publicly auditable after the fact.

6.5 The Revelation Invariant

Stated once, as the privacy stack's constitution:

The only plaintext transaction information ever produced by the network is one aggregate per block: the pair (Δ_B, R) for batches of at least 64 transactions. No

protocol message, memory dump, or ceremony output ever contains an individual δ or r in plaintext. Individual plaintexts exist only in user wallets.

Party	What they can see
Relay	Layers of onion; next hop; nothing else
Any validator, pre-reveal	Public shells: nullifiers, commitments, encrypted notes, fees, proofs, Pedersen commitments C — no amounts, no parties, no individual deltas
Block producer	Shells + the aggregate ciphertext; after ceremony: (Δ_B, R) only
Decryption committee	Their own partials; after combination: (Δ_B, R) only

Public / light client

Everything above, plus the audit equation result

6.6 Honest Limitations

- Index-layer privacy is computational and aggregate — one tier below the note layer, parameterized by $(W, B_{\min})$, and reviewed at every governance epoch (§4.6). This is precisely why the note layer is authoritative.
- Relay competence trust in cover-traffic quality (§6.2) — bounded by open specifications and stake, not eliminated.
- Noise budget: the homomorphic channel's parameters are sized for $\leq 10,000$ additions per batch with margin; the batch cap is thus a cryptographic parameter, not merely a performance one.
- Latency: ~ 1.3 s median ingress anonymity. NERV's privacy is not free; it is paid in mixing delay, once per transaction.

7. The Knowledge Layer: Forecast-Verified Self-Evolution

7.1 What Learning Is For — and What It May Never Touch

Everything before this section is deliberately static: notes, nullifiers, trees, the frozen encoder W , the circuits. This section describes the one part of NERV that moves every block: a small forecasting model, F , whose entire job is to form a public, checkable expectation of the next block's aggregate activity — and to be corrected, in public, when reality disagrees.

Two rules bind the knowledge layer absolutely:

1. Advisory only (P2). No balance, proof, receipt, or consensus outcome depends on F being *right*. Forecasts feed operations: shard pre-provisioning (§9), ceremony scheduling, fee-market baselines, anomaly telemetry. A catastrophically wrong forecast degrades operational anticipation and embarrasses the model — nothing else.
2. Exogenous targets only (P3). F is never trained on its own output, and never on anything derived from itself. Its target is the next block's revealed aggregate —

which, at the moment the forecast must be committed, is *sealed under the network's threshold key* (§6.3). This is the quiet genius of the architecture: the privacy mechanism doubles as the exam proctor. The answer key does not exist in plaintext anywhere on Earth when the forecast is due. Nobody — not the forecaster's host, not the block producer, not the decryption committee — can peek.

The per-block loop has four beats: Commit → Reveal → Score → Update.

7.2 The Forecasting Problem, Specified

Target. The model forecasts the *next* block's revealed aggregate Δ_{t+1} — not the cumulative embedding e (a running sum, which random-walks and is hostile to learning), but its per-block increment, which is roughly stationary. Differencing is elementary time-series hygiene: same information, learnable coordinates.

Because raw batch sums scale with transaction count (heteroskedastic — big batches mean big numbers for boring reasons), the target is factored into two cleaner pieces, both forecast by linear heads sharing one input:

- $\hat{\mu}_{t+1} \in \mathbb{R}^{64}$ — the predicted *mean per-transaction contribution* to the embedding
- $\hat{n}_{t+1} \in \mathbb{R}$ — the predicted *batch size*

The operational forecast of the aggregate is formed by the deterministic harness as $\hat{y}_{t+1} = \hat{n}_{t+1} \cdot \hat{\mu}_{t+1}$ (the product is computed outside the linear model — linear F cannot form cross-terms itself, so the factorization is done *for* it), and scoring is performed on the true aggregate: $r = \varphi(\Delta_{t+1}) - \hat{y}_{t+1}$, where φ is the center-and-scale transform below.

Inputs. The feature vector $x_t \in \mathbb{R}^d$ ($d \approx 530$) is built exclusively from *public history through block t* :

Block	Features	Count
-------	----------	-------

Lags	$\varphi(\Delta_t), \varphi(\Delta_{t-1}), \dots, \varphi(\Delta_{t-7})$ – eight lagged, transformed aggregates	512
Counts	n_t, n_{t-1}, n_{t-2}	3
Fees	total batch fee, fee-per-transaction, lag 1	3
Load	mempool depth, queue age	2
Cycles	coarse time-of-day and day-of-epoch buckets (public calendar, not private timing)	~ 8

The center-and-scale transform φ . The embedding accumulates modulo 2^{64} (§4.1), so a revealed Δ is a vector of residues. Before feeding residues to any learner – and before scoring – each component is mapped to its canonical signed representative in $[-2^{63}, 2^{63})$ and arithmetically shifted down by 32 bits (with saturation clamping) into model units. This is public, deterministic, bit-exact arithmetic. Two honest notes: rare per-dimension wraparound events surface as large outliers in φ -space, and the Huber

loss absorbs them — robust statistics and modular accumulators are natural allies; and $\delta = 1.0$, the Huber threshold, is defined in these φ -units throughout.

The model. F is a linear map with two heads and biases:

$$\hat{\mu} = F_{\mu} \cdot x_{\boxtimes} + c_{\mu} \quad \hat{n} = F_n \cdot x_{\boxtimes} + c_n$$

with $F_{\mu} \in Q^{64 \times d}$, $F_n \in Q^{1 \times d}$, biases in Q^{64} and Q^1 — roughly 34,500 parameters total, stored as 64-bit signed fixed-point (Q16.48). One matrix–vector product: microseconds on a phone, negligible inside a 400 ms block.

7.3 The Per-Block Loop

Step 1 — Forecast and commit (pre-reveal). Every node — validator, relay, phone — can compute $\hat{y}_{\boxplus,1}$, because $F_{\boxtimes}$ and $x_{\boxtimes}$ are pure functions of public history. The proposer assembles the block *header* first, and the header carries:

$$H_{\boxplus,1} = \text{BLAKE3}(\text{"nerv.forecast"} \parallel \text{model_version} \parallel \hat{y}_{\boxplus,1} \parallel \text{digest}(x_{\boxtimes}))$$

The header is gossiped and timestamped before any decryption partial is collected. The commit is therefore binding by protocol order, not by trust: the hash goes on record while the target is still sealed.

Step 2 — Reveal. The threshold ceremony opens the aggregate only (§6.3): the committee combines t partial decryptions, $(\Delta_{\boxplus,1}, R)$ becomes public, the audit equation $\Pi C_{\text{tx}} = \text{PedVec}(\Delta; R)$ is checked by everyone, and the embedding updates: $e_{\boxplus,1} = e_{\boxtimes} + \Delta_{\boxplus,1} \pmod{2^{64}}$ — 64 additions.

Step 3 — Score. The residual $r = \varphi(\Delta_{\boxplus,1}) - \hat{y}_{\boxplus,1}$ is computed against the *committed* forecast hash. The block's public record now contains an immutable, non-repudiable surprise vector. Loss is componentwise Huber ($\delta = 1.0$) summed over 64 dimensions; the count head is scored identically on $(n_{\boxplus,1} - \hat{n})$.

Step 4 — Update. The gradient is exactly the tensorized form a single-layer model admits — with $\psi(r)$ the blame vector ($\psi(r_{\boxtimes}) = r_{\boxtimes}$ inside the core, $\text{sign}(r_{\boxtimes}) \cdot \delta$ in the tail):

$$g_{\{F_{\mu}\}} = -\psi(r) \otimes x_{\boxtimes} \quad g_{\{c_{\mu}\}} = -\psi(r) \quad (\text{and analogously for the count head})$$

The gradient is clipped to a fixed maximum norm (the lever arm $x_{\boxtimes}$ is unbounded public data; the clip is the second half of the bound, alongside Huber's cap on blame). Then the five Adam equations run — in the fully specified fixed-point arithmetic of §7.4.

7.4 Deterministic Fixed-Point Adam (The Bit-Exactness Specification)

F is consensus-adjacent state: it is committed in every header (model_root, §4.1) and must be reproducible bit-identically by every replaying node. Floating point is therefore forbidden, and the entire optimizer is specified as integer arithmetic. This subsection is the specification that earlier designs left as debt.

State. Parameters (F_μ , F_n , c_μ , c_n), moments (m , v in the same shapes), and a step counter t — all stored as 64-bit words. m and v are *pure replay state*: recomputable from genesis, never independently attested.

Formats and operations.

Quantity	Format / rule
Parameters, moments, features	Signed Q16.48 (parameters/moments); features in saturating Q-format from φ
All products	128-bit intermediate, final round-half-even to target format
α (learning rate)	2^{-10} — an exact power of two, so the α -multiply is a lossless shift (≈ 0.000977)

ϵ 2^{-32} , exact

$\sqrt{\hat{v}}$ Integer Newton iteration; final result rounded half-even

$\beta_1 = 0.9, \beta_2 = 0.999$ Stored as Q32.32 constants; $\beta \cdot m$ products round half-even

Powers β^t Maintained *iteratively* — one rounded multiply per block — never recomputed by exponentiation

Bias correction Division by $(1 - \beta^t)$ tables, round-half-even

Clip norm A fixed Q-format constant; scaling division round-half-even

Step counter t	Optimizer steps since last reset (genesis, epoch boundary, or promotion) — <i>not</i> chain height, so resets do not corrupt the correction
------------------	---

The determinism claim. $F_{\mathbb{N}}$ is a pure function of (genesis state, the block history, the governance log). Any node replaying the chain reproduces every parameter bit-identically. There is consequently nothing to vote about: where the predecessor design asked 67% of validators to agree on a computed weight hash "within a small tolerance" (hashes admit no tolerance — the spec was unimplementable), v2.1 commits `model_root` as ordinary deterministic state, exactly like a Merkle root. Validators vote on the *executed block* (§8); the model hash rides inside it.

7.5 Why the Learning Signal Is Honest

Four structural properties, each severing one historical failure mode:

1. The target is unknown at commit time — sealed under the threshold key; the information gap is cryptographic, not procedural.
2. The target is not a function of the learner — F touches nothing that produces Δ ; W is frozen; there is no residual term that anneals toward identity.
3. The units are stationary — within an epoch, Δ is measured in fixed coordinates forever; the target's meaning does not drift as the learner learns.
4. The loss floor is strictly positive — which transactions arrive is irreducibly uncertain, so gradients never die and the training signal never self-extinguishes. (In the predecessor design, the optimum was a zero-loss fixed point at which "self-evolution" quietly stopped; a forecasting problem with genuine noise has no such exit.)

7.6 The Anomaly Alarm

Because every block pairs a *committed* forecast with a *revealed* target, the network maintains a permanent, tamper-evident record of its own surprise. The alarm rule is deliberately simple and public:

- tail-count = $\#\{j : |r_j| > \delta\}$ in φ -units, per block
- Level 1 flag: tail-count ≥ 16 (a quarter of dimensions in the Huber tail) at block t

- Level 2 flag: Level 1 sustained for 3 consecutive blocks, or gradient clipping engaged 3 consecutive blocks

Flags feed shard pre-provisioning boosts (§9), mempool and fee review, operations paging, and relay cover-traffic recalibration triggers. Flags are advisory (P8): no automatic slashing arises from a residual — a block producer can influence aggregates through inclusion choices, so residuals trigger *review*, not punishment. But because the forecast was hash-committed pre-reveal, any party steering aggregates toward "expected" values leaves statistical fingerprints in the public surprise history. Manipulation is not prevented by the alarm; it is made *visible forever*.

7.7 The Challenger Market: Useful-Work Rewards for Verified Skill

If F is deterministic replay, validators cannot be paid for "submitting good gradients" — everyone computes the identical gradient. The Useful-Work economy (funded by the pool described in §13) therefore pays for the only scarce commodity that actually exists: better models, proven out of sample.

The protocol.

1. **Propose.** Anyone posts a bonded *model proposal* committing $\text{Hash}(\text{params} \parallel \text{salt})$ of a challenger F' .
2. **Shadow-run.** For a window of K blocks (default $K = 21,600 \approx 2.4$ hours), the proposer publishes, each block, a signed hash of F' 's forecast — committing to predictions *before* each reveal, exactly like the incumbent. The challenger's parameters stay hidden during the window; only its committed predictions are public.
3. **Reveal and verify.** At window end, the proposer reveals params. Anyone checks that every committed hash matches the revealed parameters — a fraudulent run (posting hashes computed by some other model) fails this consistency check, forfeits the bond, and is slashable.
4. **Promote.** If the challenger's verified out-of-sample Huber loss beats (a) the persistence baseline $\hat{y}_0 = \Delta\bar{y}$ ("tomorrow equals today" — the minimum bar for claiming any knowledge) and (b) the incumbent F , each by a governance margin m (default 10%), the challenger becomes the canonical forecaster at the next block. The optimizer state resets (fresh $m, v, t = 0$; bias correction compensates for youth — exactly as designed).
5. **Pay.** The pool pays the promoter, proportional to the verified skill margin, capped per event.

Why this cannot be farmed. Targets are sealed at every commit (P6: out-of-sample only – in-sample loss reduction, the Goodhart currency of the predecessor design, is unreachable here by construction). Huber's δ caps the per-block extractable value of manufactured crises. Bonds price spam. And the persistence baseline means the network never pays for common knowledge.

This gives "the network gets smarter" an operational, auditable definition: *committed ex-ante forecasts of sealed targets, verified ex-post, promoted on measured skill*. Not a slogan – a stream of public, checkable events.

7.8 Epoch Boundaries

A governance epoch that rotates W (§4.7) changes the coordinate system of every future Δ . The forecaster therefore resets at every boundary: parameters re-seeded to the persistence default ($F = 0$, c set to the last pre-boundary revealed aggregate), moments zeroed, $t = 0$. It relearns the new coordinates within minutes – the fast-regime-adaptation behavior Adam + Huber is chosen for – and the reset choreography is itself deterministic replay state. The same reset applies after any challenger promotion.

7.9 What the Knowledge Layer Is – and Is Not

Is: adaptive statistics; a public expectation with a permanent scoreboard; anomaly telemetry; an operational feed; a market for verified forecasting skill.

Is not: intelligence; a consensus input; a secret; a canonical authority of any kind. The blast radius of its total failure is bounded by design: bad forecasts, bad pre-provisioning, bad alarms, public embarrassment. The ledger does not notice.

8. Consensus and Finality

8.1 What Consensus Decides

Consensus in NERV decides exactly one thing per block: the executed composite state root h_{+1} (§4.1). Everything inside that root is either deterministic arithmetic (vector additions, tree updates, audit equations) or deterministic verification (proof checks, signature checks). There is nothing approximate, nothing learned, and nothing predicted

in the subject of a vote. The forecast hash is carried in the header *as data*; a wildly wrong forecast cannot delay, fork, or invalidate a block — it can only be scored badly afterward. The separation of powers between §7 and §8 is total, and enforced by construction.

(One precision, to prevent misreading: *model_root* *is* part of the composite root — but it is deterministic replay state. Voting on it is like voting on a sum: honest validators cannot disagree, and a disagreement is evidence of a bug or fraud, not a difference of opinion.)

8.2 Block Structure and the Order of Operations

A block at height $t+1$ is produced in strict phases, and the ordering is where several security properties live:

Phase	Content	Timing note
1. Header assembly	Height, previous hash, batch commitment (Merkle root of shell hashes + sealed-ciphertext hashes), forecast hash H_{t+1} , <code>params_root</code> , <code>model_root</code> , VRF leader proof, timestamp	Gossiped before any decryption partial exists — the forecast commit and batch composition are locked first

Committee partial decryptions of ct_B

2. Aggregate ceremony (with correctness proofs), Lagrange §6.3; aggregate only
combination $\rightarrow (\Delta, R)$ public

3. Execution Audit equation check; e-update (64
adds); nullifier/commitment tree
updates; fee accounting; fraud-check ~tens of ms
all proofs (Nova-folded batch
verification, §5.5)

4. Root & vote $h_{\mathbb{M}+1}$ computed; validators attest; $\frac{2}{3}$
stake-weighted quorum certificate BLS aggregation, below
(QC) aggregates

5. Finalize & issue Block final; VDWs issuable (§10); F
update executes deterministically §7.3

8.3 Validators, Keys, and Leader Election

- Leader election: VRF-based, weighted by stake, rotating per block. The leader *orders ciphertexts and assembles headers* — that is the entire office. It cannot read transactions (§6.3), cannot reorder for profit (sums are order-invariant), and cannot alter the exam (the forecast hash is deterministic, and the batch commitment precedes the ceremony).
- Signatures: Dilithium-3 for block proposals, attestations of record, and all slashing-relevant evidence — post-quantum from genesis. BLS12-381 keys are used for *one* bounded purpose: aggregating the per-block quorum of attestation votes into a single 48-byte certificate that light clients verify against the committed validator set. BLS is not post-quantum; its role is confined to liveness and certificate compression, it protects no funds, and a migration hook to lattice-based aggregation is specified in §11.
- Reputation: a published, deterministic metric (uptime attestations, proposal validity, slashing history) computed from public data. It is explicitly *not* a learned model, and it carries no consensus weight — voting power is stake only — precisely so that no unverified signal can embed itself in the quorum. Governance may use reputation for committee and decryptor selection.

8.4 The Timing Budget

Stage	Target
Header assembly + gossip	~80 ms
Threshold ceremony (committee partials + combination, aggregate only)	~180 ms

Execution, audit equation, root computation

~20 ms

Attestation gossip + QC aggregation to $\frac{2}{3}$

~250 ms

Total to finality

≈ 530 ms (< 600 ms target)

Mixnet ingress latency (~1.3 s median, §6.2) is paid *before* the mempool and does not enter block time. Committee sizes for decryption (16–32 active decryptors per epoch, VRF-selected from the validator set) are tuned to keep the ceremony inside its budget at global scale.

8.5 Disputes: Arithmetic, Not Simulation

Because every canonical quantity is deterministic, disagreement has exactly two possible causes: faulty execution or fraud. There is no room for Monte-Carlo "simulation of outcomes" — a design flourish inherited by predecessors from an era when the state was only *approximately* computable, and retired here with it. A disputing node posts a fraud proof: the block, the incorrectly computed component (a sum, a tree update, a proof that fails, a malformed partial decryption), and the correct recomputation. Verification is a public arithmetic check; the offender is slashed per the published table (0.5–5% by severity); the whistleblower is compensated from the penalty. Dispute resolution latency: one verification, milliseconds.

8.6 Finality and Reorganization

Once a QC forms, the block is final — single-slot finality, no probabilistic confirmations, no reorg windows. VDWs (§10) are issued only against QC-finalized blocks, so a receipt

is born final and never needs revocation machinery. The chain of QCs gives light clients a <100 KB synchronization path, forever.

8.7 The Consensus Guarantee, Stated Plainly

Every value consensus votes on is either a sum anyone can recompute or a proof anyone can verify. The network's expectations, forecasts, and learned parameters exist one layer away, committed as data, scored in public, and powerless over the ledger.

9. Dynamic Sharding and Data Availability

9.1 Shards and the Root Chain

NERV launches as a single shard: one embedding, one note commitment tree, one nullifier set, one block chain. The network is its own shard. All scaling machinery below exists for the day it is needed, and none of it touches the transaction format, the circuit, or the learning loop.

When the first split occurs, the original chain persists as the root chain — a lightweight coordination layer committing only: the validator set, the relay registry, the governance log, the frozen encoder version (params_root), and one crosslink per shard block (the shard's latest finalized composite root and quorum certificate). Light clients sync the root chain (< 100 KB, forever) plus the headers of shards they care about.

Each shard owns, in full: its embedding accumulator, its nullifier set, its note commitment tree, its forecaster instance, and its validator subcommittee. Transactions are single-shard by construction (§9.4), so a shard never needs another shard's state to execute or verify anything.

9.2 Split and Merge Triggers: The Model Proposes, the Vote Disposes

Each shard runs a Shard Load Model — a small linear learner of the same family as the forecaster, but pointed at a purely exogenous target: *measured wall-clock durations* of the block pipeline phases (header gossip, ceremony, execution, attestation), against their budgets (§8.4). Features: mempool depth, batch fill rate, transactions-per-second, ceremony latency, queue age, calendar buckets. Output: an overload probability $p = \text{sigmoid}(w \cdot x)$. It trains continuously under Huber loss and clipped Adam — robustly, so that one anomalous mega-batch neither triggers a spurious split nor masks a needed one.

Wall-clock measurements are not identical across nodes, so the load model — unlike the forecaster — is *not* deterministic replay state and is *not* consensus state. This is deliberate, and the resolution respects the constitutional rule that consensus votes only on recomputable facts:

- Trigger (advisory): $p > 0.90$ sustained for 100 blocks (~40 s); a Level-1 anomaly flag (§7.6) may shorten the observation window to 25 blocks by governance parameter.
- Decision (canonical): any validator may post a SplitProposal citing the telemetry. The split occurs only on a $\frac{2}{3}$ committee vote, recorded on-chain — the vote, not the model, is the replayable event.
- Merge (symmetric): $p < 0.10$ sustained for 1,500 blocks (~10 minutes) triggers a MergeProposal under the same voting rule. Splits and merges are both first-class; shard count is a thermostat, not a ratchet.

Hysteresis (two distinct thresholds, sustained windows, and a post-event cooldown of 3,600 blocks) prevents oscillation under bursty load.

9.3 The Split Choreography — Deterministic, Noise-Free

A split finalized at height H proceeds in six exact steps. Every step is replayable from public data; no step injects randomness — a predecessor design seeded child states with differential-privacy Gaussian noise, which would have made partition events unreplayable and consensus-forking; v2.1 rejects noise wherever determinism is required.

1. Slot partition. Shard assignment of account slots extends deterministically: $\text{shard_id}(\text{slot}) = \text{leading bits of BLAKE3}(\text{"nerv.shard"} \parallel \text{slot})$. The parent's slot range splits into two disjoint children; every note, by its delivery address's slot, henceforth belongs to exactly one child.
2. Ancestry. The parent's composite root h_H is frozen as shared ancestry. Each child's note commitment tree is defined as (frozen ancestry root $\parallel$ own append-only appendix): pre-split notes remain spendable in the owning child, with membership proofs made against the frozen root — old proofs stay valid forever.
3. Nullifier inheritance. Both children inherit the *full* parent nullifier set as read-only ancestry plus their own appends. Safety over storage: a note spent before the split can never be spent again, in either child.
4. Embedding seeding. Each child's accumulator is seeded exactly: $e^0_{\text{child}} = \text{BLAKE3-XOF}_{64}(\text{"nerv.shardseed"} \parallel h_H \parallel \text{child_id})$ — a deterministic 512-byte expansion of the parent root. Thereafter each child accumulates its *own*

transactions' deltas under the same frozen W . (The embedding is an index, §4.8; reseeding it deterministically preserves continuity of tracking while committing no semantics.)

5. Forecaster reset. Each child's forecaster resets per the partition-boundary rule (§7.8): parameters to the persistence default, moments zeroed, step counter to zero. Newborn shards begin in the calibrated-birth regime Adam's bias correction exists for.
6. Settlement of the mempool. Pending sealed deltas at height H : any remainder of at least $B_{\min}$ is executed by the parent's final settlement batch before the split takes effect; any sub- $B_{\min}$ remainder is dropped without nullifier publication — wallets detect non-inclusion (no VDW arrives), and resubmit to the correct child. Nothing is lost; nothing is half-spent.

Post-split transaction shells carry a public shard tag (introduced by this machinery): the `shard_id` alone — coarse, non-linkable metadata, deliberately coarser than the private slot. Routing follows the tag.

Merge is the mirror operation: sibling slots recombine, ancestry chains concatenate, the merged accumulator is seeded deterministically from both child roots ($e^0_{\text{merged}} = \text{XOF}(\text{"nerv mergeseed"} \parallel h_A \parallel h_B)$), and the forecaster resets.

9.4 Cross-Shard Payments — Honestly Scoped

The spend circuit enforces that all input notes and all output account slots of a transaction belong to one shard (checked via a 224-entry slot→shard lookup table committed on the root chain). A payment across shards is therefore two shielded

payments — or, when atomicity is required, an HTLC-style hashlocked pair, exactly as NERV users already settle against other chains. Native cross-shard transfers (a hashlock-nullifier protocol reusing this document's primitives) are a roadmap item (§14), not a launch dependency. This is principle P8 applied to architecture: state the honest scope, defer the swamp.

9.5 Data Availability

Shard block data (headers, shells, sealed ciphertexts, revealed aggregates) is erasure-coded with Reed-Solomon $k = 5$, $m = 2$ across each shard's committee: any 5 of 7 encoded fragments reconstruct the block, so the shard tolerates 2 of 7 nodes offline with no data loss. Validators sample fragments continuously; VDWs and headers are additionally pinned to Arweave/IPFS (§10.3), giving the receipt layer a permanence independent of validator liveness.

10. Verifiable Delay Witnesses (VDWs)

10.1 What a VDW Is

The VDW is the user's permanent, offline-verifiable receipt of inclusion. Because NERV has single-slot finality (§8.6), a VDW is born final — there is no reorganization window, and therefore no revocation or replacement machinery (a predecessor design needed both; determinism deleted them).

Field	Size
tx_hash (BLAKE3 of the public shell)	32 B
shard_id, height	12 B
Merkle path of tx_hash into the block's batch commitment (~14 levels)	~448 B
Header digest	32 B
Quorum certificate (aggregated BLS signature)	48 B

Validator-set epoch reference	32 B
Core VDW	≈ 620 B
Optional: Dilithium-3 issuance attestation by a staked issuer	~2.4 KB
Attested VDW	≈ 3.1 KB

10.2 Offline Verification (< 30 ms, forever)

1. Header authenticity: the header digest matches the shard's finalized chain (via the light client's synced root chain or a trusted checkpoint).
2. Quorum: the QC verifies against the referenced validator-set epoch.
3. Inclusion: the Merkle path lands tx_hash in the header's batch commitment.
4. Participation (optional, from public data): the user's own Pedersen commitment C_{tx} appears in the batch's shell set, and the block's audit equation $\prod C_{tx} = \text{PedVec}(\Delta_B; R)$ held — public arithmetic anyone can replay (§6.3).

10.3 Permanence and Disclosure

VDWs are pinned to Arweave/IPFS within 30 seconds of finality; after five years they aggregate into daily Merkle Mountain Range buckets (≤ 1 KB proofs for century-scale retrieval). Export: QR, file, base64.

Selective disclosure, honestly stated. The public shell is already public — so sharing a raw VDW *reveals the association between you and that shell*. For compliance (exchanges, auditors, merchants), wallets derive zero-knowledge statements instead: "I hold a valid VDW and note secrets proving receipt of $\geq X$ NERV at height $\geq H$ " — 300–800 byte proofs, verifying in < 50 ms, disclosing exactly the claim and nothing else. The raw VDW is for your own archive; the derived proof is for the world.

11. Cryptographic Posture, Limitations, and Migration Hooks

11.1 The Suite

Primitive	Role	Assumption	Post-quantum?	Bound / hook
Dilithium-3 (FIPS 204)	All signatures: proposals, attestations of record, slashing	Module-LWE		Fund-grade. No bound needed

evidence, VDW
attestations

ML-KEM-768 (FIPS 203)	Relay onion layers, note encryption, keyModule-LWE transport	✓	Fund-grade
-----------------------	--	---	------------

Lattice linearly-homomorphic threshold encryption	The sealed delta channel (§6.3)	Module-LWE variant ✓	Younger engineering; mitigated by partial-decryption proofs, epoch re-keying, proactive share erasure, and the public audit equation that makes any decryption output tamper-evident. Hardening is a named roadmap dependency (§14)
--	---------------------------------	----------------------	---

BLAKE3	Hashes, commitments (notes, trees), KDF/XOF, domain separation	Hash / ARX	✓ (Grover-bounded)	Ubiquitous workhorse
SHA3-256	NIST-mandated hashing contexts	Sponge	✓	Interoperability only
ChaCha20-Poly1305	AEAD for layers and notes	Symmetric	✓	Standard
Halo2 + Nova folding	Succinct ZK proofs, recursion, batch verification	Discrete log	✗	Bounded: provides succinctness and privacy, <i>never</i> spending authority (that is Dilithium + hashes). Hook:

				STARK-family migration
				Bounded to liveness and 48-byte certificates; protects no funds. Hook: lattice-based aggregation
BLS12-381	Quorum-certificate aggregation only	Pairing	✗	
				Two bounded roles: inside the SNARK (assumption irrelevant – the outer proof is what verifies) and as public audit anchors. Hook: migrate audit commitments to hash-based vector
Pedersen vector commitments	Public audit commitments C; in-circuit range gadget	Discrete log	Binding ✗	

commitments at a
governance epoch

11.2 Migration Hooks (All Pre-Scheduled, None Panic-Driven)

Component	Today	Migrates to	Mechanism
Proof system	Halo2 + Nova	STARK family	Circuit arithmetization is hash-friendly by design (lookups, no pairings in-circuit)
QC aggregation	BLS12-381	Lattice threshold signatures	Swap at a validator-set epoch; certificates grow, finality budget re-tuned
Audit commitments	Pedersen	Hash vector commitments	Governance epoch; audit equation form unchanged

11.3 Honest Limitations (The Standing List)

1. Index-layer privacy is computational and aggregate — parameterized by (W , $B_{\min}$); formal leakage analysis is a standing governance review item, and $B_{\min}$ is the knob.
2. Relay competence trust in cover-traffic quality — priced by stake and reputation markets, bounded by open specifications, not eliminated.
3. The threshold delta channel is the protocol's youngest cryptographic component; its parameters are sized with margin, its output is self-auditing, and its hardening is externally audited before mainnet.
4. The SNARK is not post-quantum; its failure degrades privacy and cheap verification, never custody.
5. The challenger market's game theory (collusion margins, adversarial telemetry) requires incentivized-testnet stress before mainnet rewards reach full size.
6. Anomaly flags are advisory — they trigger review and pre-provisioning, never automatic punishment, because aggregates are influenceable through inclusion.

12. Governance

Who votes. Note-holders, by shielded balance at a snapshot height. Voting power is proven in zero knowledge — a *view-only attestation* reusing the spend circuit's machinery (membership + ownership + conservation, no nullifiers posted) — so governance never deanonymizes wealth.

Ballot integrity without linkability. Each wallet derives a per-referendum ballot tag $bt = \text{BLAKE3-ToField}(\text{"nerv.ballot"} \parallel \text{governance_key} \parallel \text{referendum_id})$; posting it once prevents double-voting; different referenda yield unlinkable tags. Ballot *contents* are encrypted under the epoch DKG key; tallies are revealed by committee as per-option counts — which are, pleasingly, revealed aggregates too (§6.3's invariant, reused).

What governance controls. Epoch rotation of W (with the full certificate checklist of §4.7); $B_{\min}$ and batch caps; challenger margins m and windows K ; fee curves; slashing-table amendments (supermajority + extended timelock); relay registry policy; treasury grants; migration-hook activation.

What governance can never do. Touch an individual note, reverse a finalized root, or mutate a witness. The ledger is not governable; only the parameters around it are.

Emergency powers (bounded). A supermajority committee may raise $B_{\min}$, pause challenger promotions, or extend ceremony deadlines — liveness and privacy-preserving brakes only. There is no key, anywhere, that spends someone else's money.

13. Tokenomics and the Useful-Work Economy

Fixed supply: 10,000,000,000 NERV, created entirely at genesis as shielded notes (§3.7).

No emission. No inflation.

Allocation	Share	Terms
Useful-Work Pool	72%	Released on a ~30-year declining schedule (half-life $\approx$ 6 years); unclaimed funds remain in the pool
Visionary	6%	6-month linear vesting
Code & Research	10%	12-month vesting, 3-month cliff
Early Donors	8%	12-month vesting, 6-month cliff
Treasury	4%	Governance-directed grants and audits

Fees. Declared, public, paid per §3.5's conservation equation; 100% distributed to the proposer and attesting committee (50/50). No burn is required — there is no inflation to offset.

What the pool pays for — and deliberately does not. Because the forecaster is deterministic replay, there are no per-block "gradient wages" to sell: every honest node computes the identical gradient, and paying for it would be a subsidy in costume. The pool pays only for verified, scarce skill and safety:

Payment	Trigger	Bound
Promotion rewards	Challenger beats persistence and incumbent baselines out-of-sample (§7.7)	Proportional to verified margin; capped per event
Fraud bounties	Valid fraud proofs (§8.5)	From slashed amounts
Security & research grants	Leakage analyses, audits, circuit bounties	Governance-approved

The anti-Goodhart stack, restated as economics: out-of-sample targets only, baseline-or-nothing, Huber-capped crisis value, bonds against spam. The pool cannot be farmed by manufacturing events, because it never pays for in-sample anything.

14. Implementation Roadmap

Phase	Scope	Exit criteria
P0 (weeks 1–3)	Custody layer + circuits: notes, nullifiers, NCT, spend and delta circuits, fixed-point conformance harness	Circuit test vectors; constraint budget $\leq 55K$
P1 (weeks 4–6)	libp2p/GossipSub, BFT pipeline, VRF leaders, QC aggregation, DKG tooling	5-node consensus at < 600 ms finality

P2 (weeks 7–8)	Privacy stack: relay registry, PQ-Sphinx client + relay, threshold delta channel, partial-decryption proofs, audit-equation checker	End-to-end sealed-delta block with public audit
P3 (weeks 9–10)	Knowledge layer: φ -transform, fixed-point Adam (differentially tested against a float oracle), commit–reveal–score–update, anomaly flags	Public testnet with dashboards; determinism CI (identical hashes across x86/ARM/RISC-V)
P4 (quarter 2)	External audits (circuits, DKG, threat model); threshold-channel hardening; formal leakage analysis of (W, B _{min}); Audit remediation complete challenger-market stress on incentivized testnet; split/merge drills	

	Universal SRS + DKG ceremonies;	
P5 (genesis)	mainnet launch; Arweave pinning;	—
	monitoring	

Named dependencies, honestly: the lattice threshold channel's maturity (P4), the leakage analysis (P4), and challenger-market game-theory review (P4). NERV v2.1 launches when these close — not before.

15. Conclusion

NERV v2.1 stands on three pillars, each inherited from a failure it refuses to repeat:

1. Identity–Knowledge Separation — the encoder W is a frozen constitution; the forecaster F is a living press. The network evolves what it knows, never what it is.
2. Aggregate-Only Revelation — no individual delta is ever plaintext to any node; committees open sums, and one public equation audits them. Privacy by linearity, not by hardware.
3. Forecast-Verified Self-Evolution — every block pairs a committed prediction with a sealed-then-revealed target; learning is exogenous by cryptography, scored in public, and rewarded only out of sample.

Money lives in cryptography. The state fingerprint lives in linear algebra. Expectations live in statistics, on a leash. The code can be written with today's tools; the assumptions

it rests on are named; and the things it does not claim are written down next to the things it does.

Appendix A: End-to-End Transaction Lifecycle

A complete walkthrough: Alice pays Bob v nano-NEUV. Business process first, formalism second, for every phase. Section references appear inline.

Phase 1: Client-Side Construction

Business process. Alice opens her wallet, enters Bob's address, the amount v , approves by biometric, and the wallet does everything else offline. It scans its local store of unspent notes, selects inputs covering $v + \text{fee}$, builds Bob's note and her change note, computes the transaction's effect on the public state index, proves everything in zero knowledge, seals that effect under the network's collective key, and wraps the result for anonymous routing. Nothing about amounts, parties, or notes leaves the device in the clear.

Step 1.1 — Note selection. The wallet picks inputs $I = \{n_1, \dots, n_k\}$ with $\sum v_i \geq v + \text{fee}$, and constructs outputs: Bob's note (value v) and change ($\sum v_i - v - \text{fee}$).

Step 1.2 — Feature vector and delta. The wallet builds $\Delta S \in \mathbb{Z}^{256}$ (§4.2): $\pm$ values at the account slots of sender and recipients, plus the volume, fee, count, log-magnitude, and type/time rails; checks admissibility (≤ 6 active slots); and computes the delta under the current frozen encoder:

$$\delta_{tx} = W_{\tau} \cdot \Delta S \pmod{2^{64}}$$

where τ is the current weight-version tag, fetched from the latest header.

Step 1.3 – Proofs. The wallet generates $\pi = (\pi_{\text{spend}}, \pi_{\delta})$ (§5.1–5.2): membership, ownership, nullifier derivation, conservation, ranges, feature admissibility, delta correctness, $\delta \neq 0$, and the Pedersen vector commitment:

$$C = \text{PedVec}(\delta_{tx}; r) = \sum \delta_{tx,j} \cdot G_j + r \cdot H \text{ with fresh blinding } r.$$

Step 1.4 – Sealing. The wallet encrypts the pair $(\delta_{tx}; r)$ under the network's DKG public key with the linearly-homomorphic scheme, producing ciphertext ct (§6.3).

Step 1.5 – Shell assembly. Public shell: nullifiers $nf_1 \dots nf_k$, output commitments $cm_1 \dots cm_m$, encrypted notes, declared fee, expiry, weight tag, (shard tag, post-split), π , C , ct .

Conservation (proven in-circuit): $\sum \text{inputs} = \sum \text{outputs} + \text{fee}$.

Phase 2: Mixnet Ingress

Business process. The wallet picks five staked relays from the on-chain registry (VRF-seeded), wraps the shell in five post-quantum onion layers, and submits to the first relay. Each relay peels one layer, holds the packet for an exponential delay, and forwards. Dummy packets shaped by the relay's local traffic model flow alongside. Median transit: ~ 1.3 s.

Formalism. Per hop i : $(ss_i, ct_i) = \text{ML-KEM-768.Encapsulate}(pk_i)$; $k_i = \text{BLAKE3-KDF}(ss_i, \text{"sphinx"} \parallel i)$; $\text{Layer}_i = \text{ChaCha20-Poly1305.Encrypt}(k_i, \text{next_hop} \parallel \text{Layer}_{i+1})$. Per-relay delay $\sim \text{Exp}(\mu = 100 \text{ ms})$; packets uniform 8 KB; final relay submits the shell to the threshold mempool. *Protected: the link between Alice's IP and this shell — nothing else, and nothing more is needed (§6.2).*

Phase 3: Threshold Mempool

Business process. The shell gossips among validators. Every check is public or zero-knowledge: format, fee sufficiency, proof verification, nullifier freshness (a plain set lookup — nullifiers are public tags), anti-spam limits. No node — including every validator — can read δ_{tx} : it exists only as ct . The shell waits for batching.

Formalism. Admission: π verifies $\bigwedge \{nf_i\} \cap \text{NullifierSet} = \emptyset \wedge \text{fee} \geq \text{floor}$. Batch assembly: $B = \{\text{shell}_1, \dots, \text{shell}_n\}$, $B_{\min} = 64 \leq n \leq 10,000$ (§6.4). Batch commitment $M_B = \text{MerkleRoot}(\{\text{hash}(\text{shell}_i)\})$.

Phase 4: Forecast Commit — Sealing the Exam

Business process. Before anything is decrypted, the proposer assembles the block *header* and gossips it. Embedded in that header is a hash of the network's prediction of this block's aggregate — computed by every node deterministically from public history. From this instant, the exam is locked: the target is still encrypted, the prediction is hash-committed, and neither side can move.

Formalism (§7.2–7.3). Features x_{f} (eight lagged transformed aggregates, counts, fees, load, calendar). Heads: $\hat{\mu} = F_{\mu} \cdot x_{\text{f}} + c_{\mu}$, $\hat{n} = F_n \cdot x_{\text{f}} + c_n$; harness forecast $\hat{y}_{t+1} = \hat{n} \cdot \hat{\mu}$.

Header carries:

$H_{t+1} = \text{BLAKE3}(\text{"nerv.forecast"} \parallel \text{model_version} \parallel \hat{y}_{t+1} \parallel \text{digest}(x_{\text{f}}))$ — published *before* any decryption partial exists.

Phase 5: The Aggregate Ceremony

Business process. The producer — whose entire office is ordering ciphertexts — homomorphically sums the sealed deltas into one batch ciphertext. The decryption committee (VRF-selected, threshold $t = \lceil 2n/3 \rceil$) contributes partial decryptions, each with a proof of correctness. The partials combine to reveal exactly one thing: the aggregate pair. Everyone checks one public equation against the commitments the wallets published hours or milliseconds ago.

Formalism (§6.3). $\text{ct}_B = \sum_{\text{ct} \in B} \text{ct}$ (homomorphic sum). Partial P_i with NIZK of correct decryption; combination yields (Δ_B, R) . Public audit:

$\prod_{\text{tx} \in B} C_{\text{tx}} = \text{PedVec}(\Delta_B; R)$ — holds $\Rightarrow$ the reveal is complete and honest. Failure $\Rightarrow$ quarantine, individual audit of partials, slashing.

Phase 6: Execution and State Transition

Business process. With the aggregate public and audited, execution is arithmetic: the embedding advances by 64 additions, nullifiers enter the spent set, new commitments enter the tree, fees are credited, and all 10,000 proofs verify as one folded check.

Formalism. $e_{i+1} = e_i + \Delta_B \pmod{2^{64}}$; nullifier and commitment tree roots update;

Nova-folded batch verification of $\{\pi_i\}$; composite root:

$h_{i+1} = \text{BLAKE3}(e_{i+1} \parallel \text{nct_root} \parallel \text{nullifier_root} \parallel \text{params_root} \parallel \text{model_root} \parallel \text{validator_root})$ (§4.1).

Phase 7: Scoring and the Update — the Network Learns

Business process. Now the committed forecast is graded. The residual between prediction and revealed reality becomes this block's public, permanent *surprise record*. Ordinary blocks teach the forecaster gently; extraordinary blocks trip the anomaly flag and teach it cautiously. The forecaster's weights update by five deterministic equations, identically, on every node on Earth.

Formalism (§7.3–7.4, 7.6). $r = \varphi(\Delta_{i+1}) - \hat{y}_{i+1}$; $\text{loss} = \sum \text{Huber}(r_i; \delta = 1.0)$; $\text{tail-count} = \#\{j : |r_j| > \delta\} \rightarrow \text{flags}$. Gradient $g = -\psi(r) \otimes x_i$ (clipped); Adam in Q16.48 fixed point: $m \leftarrow \beta_1 m + (1 - \beta_1)g$; $v \leftarrow \beta_2 v + (1 - \beta_2)g^2$; $\hat{m} = m / (1 - \beta_1^t)$; $\hat{v} = v / (1 - \beta_2^t)$; $F \leftarrow F - 2^{-10} \cdot \hat{m} / (\sqrt{\hat{v}} + 2^{-32})$. New `model_root` rides the next header — as replay state, never as a vote.

Phase 8: Consensus and Finality

Business process. Validators attest to the executed root — a value anyone can recompute — and the attestations aggregate into one 48-byte certificate. If anyone disagrees, they post a fraud proof: a specific sum, tree update, or proof, recomputed publicly. No simulations, no predictions under consensus, no tolerance windows.

Formalism (§8). $\frac{2}{3}$ stake-weighted quorum on h_{+1} ; QC = BLS aggregate; finality ≈ 530 ms from header. Disputes: fraud proof $\rightarrow$ verify $\rightarrow$ slash (0.5–5%) $\rightarrow$ compensate the reporter.

Phase 9: Shard Operations (Conditional)

Business process. Most blocks, nothing happens here. Under sustained overload (or sustained quiet) the shard's load model crosses a threshold, a validator proposes a split (or merge), the committee votes, and — if approved — the deterministic choreography of §9.3 executes: slots partition, ancestry freezes, accumulators reseed, forecasters reset. The ledger's history is untouched by any of it.

Phase 10: Receipt and Settlement

Business process. Alice's wallet receives a ~ 620 B core VDW, verifies it offline in under 30 ms, and marks the notes spent. Bob's wallet trial-decrypts the new note with its detection keys, discovers the value, and archives the note. The proposer and committee receive the fees. The payment is permanently confirmed — and, if either party ever needs to prove it to a third party, they derive a zero-knowledge statement from their VDW rather than revealing the linkage.

Formalism. VDW verification: header chain $\rightarrow$ QC $\rightarrow$ Merkle path $\rightarrow$ (optional) C_tx participation in the audited batch (§10.2). Economic settlement completes the conservation identity opened in Phase 1.

One Transaction, Ten Phases

Phase	Guardian	Guarantee delivered
1 Construct	Circuits + frozen W	Correctness provable, privacy preserved at the source
2 Route	PQ-Sphinx	Sender unlinked from submission
3 Pool	Public checks	Invalid txs die before inclusion
4 Forecast	Threshold seal	The exam is locked before the answer exists
5 Reveal	Aggregate-only ceremony No individual delta ever plaintext, anywhere	

6 Execute	Linear algebra	$O(1)$, exact, replayable state update
7 Learn	Huber + Adam	Bounded blame; public surprise; honest adaptation
8 Finalize	BFT on executed roots	~530 ms finality on recomputable facts
9 Split/Merge	Vote, not model	Scaling without semantic change
10 Receipt	Born-final VDW	Permanent, offline, private proof of inclusion